

01 - Review stato attuale progetto

Alessandro Gardini

`alessandro.gardini7@studio.unibo.it`

Lorenzo Rossi

`lorenzo.rossi50@studio.unibo.it`

Tirocizio - Università di Bologna - Campus di Cesena

10 giugno 2026

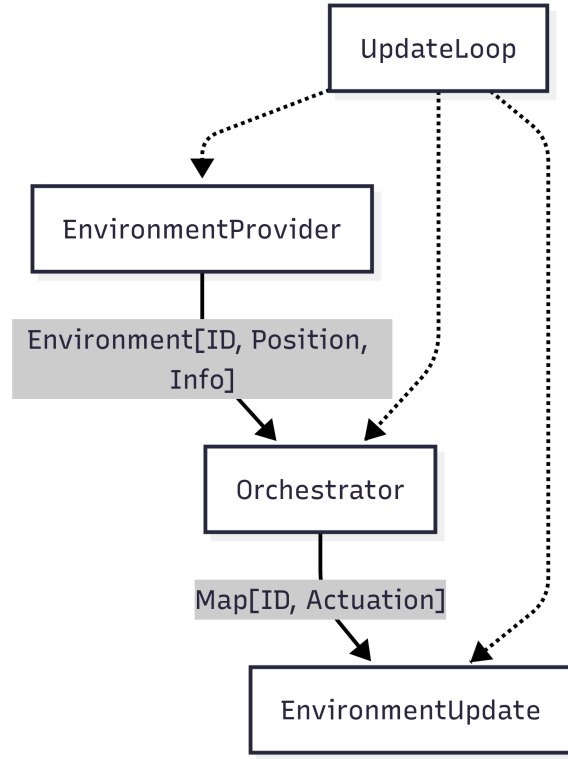


Figura 1: Flow per ogni round all'interno di UpdateLoop

1 aggregate-runtime

Modulo che raccoglie posizioni e neighborhood di ogni robot e calcola il suo vettore spostamento, pubblicato poi in `robots/{robot_id}/move`.

Si divide in principalmente in due sottomoduli `core` e `demo`.

1.1 Core

`Core` definisce il ciclo di controllo astratto del sistema — leggi l'ambiente, calcola le attuazioni, applicale — parametrizzando ogni aspetto dominio-specifico (`ID`, `Position`, `Info`, `Actuation`). Grazie a questi type parameter, l'intera pipeline (1) `EnvironmentProvider` → `Orchestrator` → `EnvironmentUpdate` può essere riusata per qualsiasi sistema di robot, indipendentemente da come sono identificati, dove si trovano, cosa percepiscono e come si muovono.

1.2 Demo

`Demo` è la realizzazione concreta del ciclo astratto definito in `core`, specializzata per uno sciame di robot fisici coordinati tramite MQTT e aggregate computing. Fissa i type parameter lasciati aperti da `core` — `ID` = `Int` (ID del marker ArUco), `Position` = (`Double`, `Double`), `Info` = `Map[String, Any]`, `Actuation` = `Rotation` | `Forward` | `Stop` | `NoOp` — e li collega a tre componenti concrete: `MqttProvider` che costruisce dinamicamente l'ambiente ascoltando i topic MQTT (posizioni, vicinati, configurazione), `AggregateOrchestrator` che esegue un round ScaFi per ogni robot producendo un'attuazione, e `RobotUpdateMqtt` che traduce quell'attuazione in comandi motore pubblicati su MQTT. I programmi ScaFi in `scenarios/` — `PointTheLeader`, `VFormation`, `CircleFormation` e altri — defi-

niscono i comportamenti collettivi dello sciame, selezionabili a runtime tramite il topic `sensing` senza riavviare il sistema. Il punto di ingresso `ResearchNightDemos` assembla tutti i pezzi e avvia il loop.

1.3 Considerazioni

- Non abbiamo capito perchè `AggregateOrchestrator` sia dentro a `core`. Non sarebbe meglio spostarlo dentro a `demo`.
- Non abbiamo capito al meglio le motivazione dell'uso dei type parameters e il loro funzionamento (Compenion objects, ecc.).
- Abbiamo valutato la fattibilità di realizzare offloading. Al momento l'`MqttProvider` riceve e riempie il `world` (mappa al suo interno) con tutti i messaggi che riceve da `robots/+position` e `robots/+neighbors`. A noi potrebbe bastare, durante la chiamata a `provide`, restituire un `Environment` dove non appaiono in `nodes` gli `id` dei robot che stanno attualmente computando localmente. Quindi andare a modificare `MqttEnvironment.nodes`.

2 dashboard

Non abbiamo approfondito il funzionamento di questo modulo. Ci siamo limitati a guardare i collegamenti che questa ha con l'esterno. Nell'immagine 3 vengono mostrate graficamente le connessioni.

2.1 Considerazioni

- I vari meccanismi di *off-loading* potrebbero essere mostrati graficamente, soprattutto se questi fossero dinamici. In questo caso potrebbe essere interessante mostrare all'utente quali macchine stanno attualmente computando i movimenti in *locale* e quali su *server*.
- Potrebbe essere possibile fare in modo che l'utente possa dinamicamente decidere su quali macchine computare *localmente* e quali *off-loadare* su *server*.

3 aruco-detector

Modulo per la cattura in tempo reale della posizione dei robot. Dentro a `main.py` vengono importate le librerie `OpenCV` e `paho-mqtt`. Avviata l'applicazione viene costantemente visionato il camera feed e attraverso `OpenCV` e `ArUcoRobotPoseEstimator`, definito in `estimator.py`, vengono aggiornate le posizioni e l'angolazione per ogni robot. A seguito dell'aggiornamento, per ogni robot, vengono pubblicate su MQTT le nuove informazioni sul topic `robots/{robot_id}/position`.

È prevista una modalità di debug per mostrare a schermo il funzionamento di `OpenCV`. Sono prevesiti calibrizioni varie per la camera.

3.1 Considerazioni

- Non abbiamo capito perchè si utilizza `numpy`. `Matplotlib` non viene usata.
- Non abbiamo trovato `calibration.py` per calibrare la camera.

4 neighborhood-system

Modulo per il calcolo delle neighborhood di ogni singolo robot. Dentro a `main.py` vengono importate le librerie `Flask` e `paho-mqtt`. Il modulo di registra ai topic `robots/+position` ricevendo le posizioni dei robot. Ogni volta che viene pubblicato una posizione per un determinato robot, viene ricalcolata la sua neighborhood. Il calcolo può essere eseguito in due modalità differenti (`neighborhood_type`): *full*, in cui tutti i robot sono considerati neighbor, e *radius*, in cui è possibile specificare un raggio (`radius_value`) entro il quale i robot vengono considerati neighbor. La nuova neighborhood viene pubblicata su MQTT sul topic `robots/{robot_id}/neighbors` (nel caso il set di neighbors risulti diverso dal precedente).

Per modificare `neighborhood_type` e `radius_value` vengono esposti con due endpoint http con `Flask`, uno REST e uno che restituisce un form HTML, per debug.

4.1 Considerazioni

- Non sono controllati gli errori per `neighborhood_type` durante la modifica.
- Non capiamo se l'endpoint "/" serve ancora o serve solo per debug.
- Si potrebbe considerare di utilizzare mqtt in sostituzione di HTTP per la comunicazione con la dashboard.
- Abbiamo una minima esperienza nell'utilizzo di `FastAPI`, `aiomqtt` e in generale di `asyncio`. Nel caso serva su il progetto di Iot per il professor Ricci è qui .

5 Considerazioni sui tirocini

5.1 Off-loading

L'off-loading essendo realizzabile a vari livelli avevamo pensato ad alcune soluzioni. Una soluzione dove è l'utente a decidere quando usare l'hardware dei robot attraverso la dashboard ed essi rispondano nel caso siano in condizioni di poter accettare la richiesta (nel caso di batteria troppo bassa rifiuterebbero la richiesta), un'altra soluzione dove i droni partirebbero in automatico in offloading e restituirebbero l'esecuzione dei processi solo in casi di criticità, dove quindi l'offloading risulta comunque dinamico ma non sotto il controllo dell'utente. Abbiamo dei dubbi riguardo alla dimensione del carico di lavoro da scaricare sui droni, i robot forniti di hardware per l'offloading dovrebbero eseguire il lavoro solo per se stessi o anche per altri droni in maniera tale da alleggerire ulteriormente il peso sul server centrale.

5.2 Possibilità di robot che comunicano con protocolli diversi da MQTT

Se non abbiamo capito male, il compito di questa parte sarebbe rendere possibile la comunicazione ai robot con protocolli diversi da MQTT. Abbiamo pensato che questo obiettivo possa essere raggiunto creando un server capace di parlare, da un lato con una serie di protocolli, come ad esempio HTTP e dall'altro lato, quello interno all'applicativo, con MQTT. Questo server fungerebbe come una sorta di *man in the middle* capace di inoltrare i messaggi al *broker* MQTT interno. I robot volenterosi di usare ancora MQTT continuerebbero ad interfacciarsi con il *broker* interno. A seguito una rappresentazione astratta del sistema.

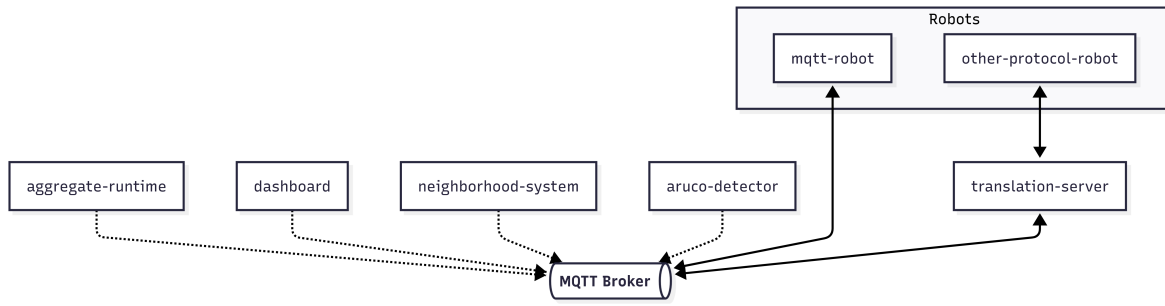


Figura 2: Possibile soluzione

6 Considerazioni generali

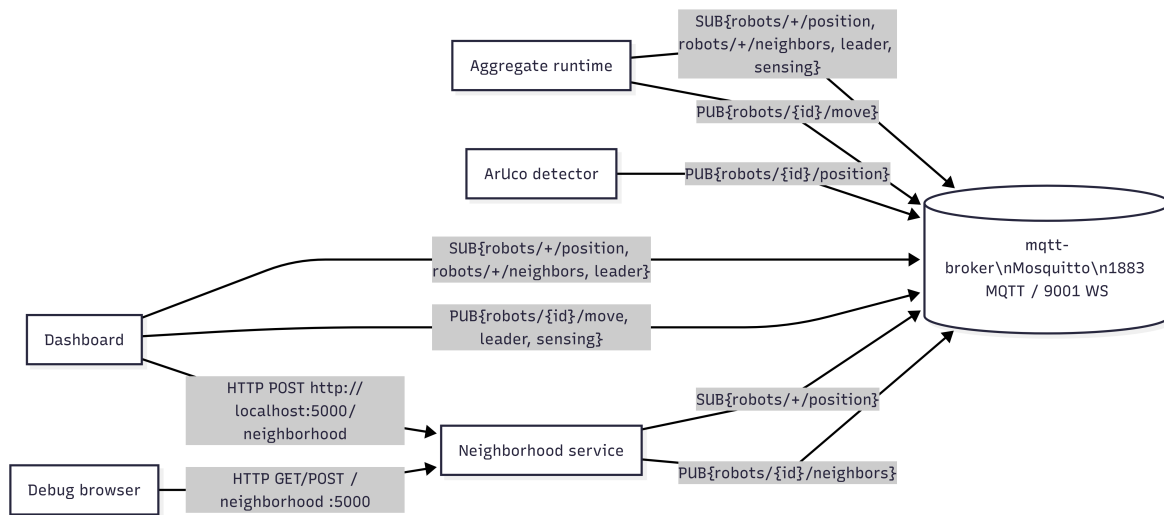


Figura 3: Connessioni

- In figura 3 sono rappresentate tutte le connessioni attuali tra i moduli del progetto.
- Attualmente le configurazioni (`build.sbt`, `pyproject.toml`, `package.json`, `vite.config.ts`) sono tutte nella root del repository. Avrebbe senso spostarle nelle rispettive sottocartelle di modulo? Potrebbe anche allineare *neighborhood-system* ad usare `uv`, come già fa *aruco-detector*.
- È disponibile il codice dei robot fisici? Nel caso, nel nostro percorso con il professor Ricci abbiamo utilizzato PlatformIO per programmare un Arduino e un ESP32.